Topics and Guideline

Topics Covered

- Trees
    - Binary search tree (insertion, deletion)
    - Inorder, preorder, postorder
- Graphs
    - BFS, DFS, Prim, Kruskal, Single source shortest path (Dijkstra)
- Hashing

Highly Recommended - revisit the assignment materials

Number of questions - 3

Time allowed - 2 hours

Programming Language - C or C++ (Code template will provide in C)

Development Environment - Codern or VS code

Exam Schedule - Section 31 - 10:00-12:00 Section 32 - 13:30-15:30

Example Questions-Graph

Problem 1 (Checked)

You are given a directed graph with n nodes labeled from 0 to n - 1 and a list of directed edges. Each edge is represented as: u v which means: there is a directed edge from node u to node v. The **indegree** of a node is the number of edges coming **into** that node.

Your task is to return the indegree of a given node target.

int getIndegree(int n, int edges[][2], int m, int target);

Input Format

- First line: two integers n and m
- Next m lines: two integers u and v
- Last line: integer target

Output Format

- Print a single integer: indegree of target

Example 1

Input
4 4
0 1
0 2
1 2
2 3
2
Output 2

Explanation

Edges going into node 2: 0 → 2 and 1 → 2 So indegree = **2**

Example 2

Input
5 3
1 0
2 0
3 0
0

Output 3

Constraints

- 1 <= n <= 100
- 0 <= m <= 500
- 0 <= target < n

C Template

```
#include <stdio.h>

#define MAXM 500

int getIndegree(int n, int edges[][2], int m, int target) {
    int indegree = 0;
        // TODO
    return indegree;
}

int main() {
    int n, m;
    int edges[MAXM][2];

    scanf("%d %d", &n, &m);
```

```
    for (int i = 0; i < m; i++) {
        scanf("%d %d", &edges[i][0], &edges[i][1]);
    }

    int target;
    scanf("%d", &target);

    int result = getIndegree(n, edges, m, target);

    printf("%d\n", result);

    return 0;
}
```

Problem 2 (Checked)

You are given a weighted directed graph with n nodes labeled from 0 to n - 1 and a list of edges. Each edge is represented as: u v w which means: there is a directed edge from node u to node v and the cost to travel is w

Your task is to compute the **minimum distance from a starting node start to a target node target**.

If the target node cannot be reached, return: -1

int shortestDistanceToTarget(int n, int edges[][3], int m, int start, int target);

Input Format

- First line: two integers n and m
- Next m lines: three integers u, v, w
- Last line: two integers start and target

Output Format Print a single integer: the minimum distance

Example 1

Input
5 6
0 1 2
0 2 4
1 2 1
1 3 7
2 4 3
3 4 1
0 4

Output 6

Explanation

Best path:

0 → 1 → 2 → 4
cost = 2 + 1 + 3 = 6

Example 2

Input
4 2
0 1 1
1 2 2
0 3

Output -1

Explanation Node 3 cannot be reached from node 0.

Example 3

Input
3 3
0 1 5
1 2 2
0 2 10
0 2

Output 7

Explanation

Better path: 0 → 1 → 2 = 5 + 2 = 7 instead of direct 10

Constraints

- 1 <= n <= 100
- 0 <= m <= 500
- 0 <= w <= 1000

Full C Template (Student Version)

```
#include <stdio.h>
#include <limits.h>

#define MAXN 100
#define MAXM 500

int shortestDistanceToTarget(int n, int edges[][3], int m, int start, int target) {
    int dist[MAXN];
    int visited[MAXN] = {0};

        //TODO

    return -1;
}
```

```
int main() {
    int n, m;
    int edges[MAXM][3];

    scanf("%d %d", &n, &m);

    for (int i = 0; i < m; i++) {
        scanf("%d %d %d", &edges[i][0], &edges[i][1], &edges[i][2]);
    }

    int start, target;
    scanf("%d %d", &start, &target);

    int result = shortestDistanceToTarget(n, edges, m, start, target);

    printf("%d\n", result);

    return 0;
}
```

Problem 3 (Checked)

You are given a directed graph represented as an adjacency matrix of size `n × n`.

- `adj[i][j] = 1` means there is an edge from node `i` to node `j`
- `adj[i][j] = 0` means no edge

Starting from node `start`, perform a **Depth-First Search (DFS) using a stack.**

Task

- If `target` is reachable, return the list of visited nodes in DFS order **until target is found**
- If `target` is NOT reachable, return:

-1

Always visit neighbors in **increasing index order**

Function to Complete

int dfsStack(int n, int adj[][MAX], int start, int target, int result[]);

Input Format

- First line: integer `n`
- Next `n` lines: adjacency matrix
- Last line: `start target`

Output Format

- If reachable: node1 node2 node3 ...

- If not reachable: -1

Example 1

Input

5
0 1 0 0 0
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1
0 0 0 0 0
0 3

Output

0 1 2 3

Example 2

Input

4
0 1 0 0
0 0 0 0
0 0 0 1
0 0 0 0
0 3

Output -1

Example 3

Input

5
0 1 1 0 0
0 0 0 1 0
0 0 0 0 1
0 0 0 0 0
0 0 0 0 0
0 4

Output 0 1 3 2 4

C template

```
#include <stdio.h>

#define MAX 100

int dfsStack(int n, int adj[][MAX], int start, int target, int result[]) {
    int visited[MAX] = {0};
    int stack[MAX];
    int top = -1;
    int size = 0;


    while (top != -1) {

    }

    // TODO: return -1 if target not found
}

int main() {
    int n;
    int adj[MAX][MAX];
```

```
    int result[MAX];

    // input number of nodes
    scanf("%d", &n);

    // input adjacency matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }

    int start, target;
    scanf("%d %d", &start, &target);

    int size = dfsStack(n, adj, start, target, result);

    if (size == -1) {
        printf("-1\n");
    } else {
        for (int i = 0; i < size; i++) {
            printf("%d", result[i]);
            if (i < size - 1) printf(" ");
        }
        printf("\n");
    }

    return 0;
}
```

Problem 4

You are given `n` cities numbered from `0` to `n - 1` and a list of roads.
Each road is represented as three integers `[u, v, w]` where:

- `u` and `v` are two different cities
- `w` is the cost to connect them

Your task is to find the **minimum total cost** required to connect all cities so that every city can reach every other city. Return the total cost of the **Minimum Spanning Tree (MST)**. If it is not possible to connect all cities, return `-1`.

Example 1

Input:

n = 4
edges = [[0,1,1],[0,2,4],[1,2,2],[1,3,5],[2,3,3]]

Output: 6

Explanation:

Choose these edges:

- 0 → 1 with cost 1
- 1 → 2 with cost 2
- 2 → 3 with cost 3

Total cost = 1 + 2 + 3 = 6

Example 2

Input:

n = 4

edges = [[0,1,2],[2,3,3]]

Output: -1

Explanation:

The graph is disconnected, so it is impossible to connect all cities.

Constraints

- `1 <= n <= 100`
- `0 <= edgesSize <= n * (n - 1) / 2`
- `edges[i].length == 3`
- `0 <= u, v < n`
- `u != v`
- `1 <= w <= 10^4`

C Template

```
#include <stdio.h>
#include <limits.h>

int minCostToConnectCities(int n, int edges[][3], int edgesSize) {


    return totalCost;
}

int main() {
    int n = 4;
    int edges[][3] = {
```

```
        {0, 1, 1},
        {0, 2, 4},
        {1, 2, 2},
        {1, 3, 5},
        {2, 3, 3}
    };
    int edgesSize = sizeof(edges) / sizeof(edges[0]);

    int result = minCostToConnectCities(n, edges, edgesSize);
    printf("Minimum cost: %d\n", result);

    return 0;
}
```

Example Question-Hashing

Linear Probing Problem (Checked)

You are given a hash table of size m and a list of n keys.

The hash function is:

h(k) = k % m You must insert all keys into the hash table using **linear probing**.

Rules

- If the computed index is occupied, move to the next index:

(h(k) + 1) % m

- Repeat until an empty slot is found
- If the table is full, ignore the remaining keys
- Empty slots should contain -1

Function to Complete
void insertHash(int table[], int m, int keys[], int n);

Input Format
m n
k1 k2 k3 ... kn

Output Format

Print the final hash table:

table[0] table[1] ... table[m-1]

Example 1

Input
7 3 // m = 7, n =3
7 12 17

Output 7 -1 -1 17 -1 12 -1

Explanation
7 % 7 = 0 → insert at index 0
12 % 7 = 5 → insert at index 5
17 % 7 = 3 → insert at index 3

Example 2

Input
5 4
1 6 11 16

Output -1 1 6 11 16

Explanation
1 % 5 = 1 → insert at index 1
6 % 5 = 1 → collision → index 2
11 % 5 = 1 → collision → index 3
16 % 5 = 1 → collision → index 4

Example 3

Input
5 3
4 9 14

Output 9 14 -1 -1 4

Explanation
4 % 5 = 4 → index 4
9 % 5 = 4 → collision → index 0 because 0 is empty (4+1=5 is overflow)
14 % 5 = 4 → collision → index 1

Constraints

- 1 <= m <= 100
- 1 <= n <= m
- Keys are non-negative integers

C Template

```
#include <stdio.h>

#define MAX 100

void insertHash(int table[], int m, int keys[], int n) {


 // TODO
}

int main() {
    int m, n;
    int table[MAX];
    int keys[MAX];

    scanf("%d %d", &m, &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &keys[i]);
    }
```

```
    insertHash(table, m, keys, n);

    for (int i = 0; i < m; i++) {
        printf("%d", table[i]);
        if (i < m - 1) printf(" ");
    }
    printf("\n");

    return 0;
}
```

Problem-2

You are working at a zoo 🦁 and some monkeys 🐒 are stealing bananas 🍌.

Each monkey has an ID number. You are given a list of monkey IDs in the order they visited the banana storage. If a monkey appears **more than once**, it means the monkey came back and stole bananas again! Your task is to detect the **first monkey who steals bananas twice**.

Task

Return:

- the **first repeated monkey ID**
- if no monkey repeats, return: -1

Function to Complete

int firstRepeating(int arr[], int n);

Input Format

n
a1 a2 a3 ... an

Output Format

MonkeyID (first repeating)
or

-1

Example 1

Input

6
5 3 2 5 4 2

Output 5

Explanation

Monkey 5 appears again first → 🍌 thief caught!

Example 2

Input

5
1 2 3 4 5

Output -1

Explanation

No monkey repeats → all innocent 😇

Example 3

Input

7
7 8 9 8 10 7 6

Output 8

Constraints

- `1 <= n <= 100`
- `0 <= arr[i] <= 1000`

Test Case 1

5
2 2 3 4 5

Output: 2

Test Case 2

4
9 8 7 6

Output: -1

Test Case 3

6
1 3 5 3 1 5

Output: 3

C Template (Student Version)

```
#include <stdio.h>

#define MAX 100
#define HASH 1001   // IDs from 0 to 1000

int firstRepeating(int arr[], int n) {
 // TODO

    return -1;
}

int main() {
    int n;
```

```
    int arr[MAX];

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int result = firstRepeating(arr, n);

    printf("%d\n", result);

    return 0;
}
```

Example Questions-Trees

Question 1: BST – Find In-order Successor of a Given Node

Problem

You are given the root of a Binary Search Tree and an integer `key`.
Write a function to **find and print the in-order successor** of that node.

Function Prototype

```
void findSuccessor(struct Node* root, int key);
```

Output Rules

- Print the **in-order successor value**
- If no successor exists, print:

No successor

Example

Input BST (already constructed):

50 30 75 20 40 60 90

Input:

key = 40

Output:

50

Constraints

- Do NOT use arrays
- Use BST properties only
- Do not modify the tree

```
#Template
#include <stdio.h>
#include <stdlib.h>

struct Node {
```

```
};

/* Create a new BST node */
        //TODO

/* Insert into BST */
struct Node* insert(struct Node* root, int value) {
        //TODO
}

/* Find minimum node in a subtree */
struct Node* findMin(struct Node* root) {
   while (root != NULL && root->left != NULL) {
      root = root->left;
   }
   return root;
}

/* Find and print in-order successor */
void findSuccessor(struct Node* root, int key) {
   struct Node* current = root;
   struct Node* successor = NULL;

   while (current != NULL) {
      if (key < current->data) {
         successor = current;
         current = current->left;
      } else if (key > current->data) {
         current = current->right;
      } else {
         /* Node found */
         if (current->right != NULL) {
            successor = findMin(current->right);
         }
         break;
      }
   }
   if (current == NULL) {
      printf("Key not found\n");
   } else if (successor == NULL) {
```

```
        printf("No successor\n");
    } else {
        printf("%d\n", successor->data);
    }
}
/* Free tree memory */
 //TODO
}
int main() {
    struct Node* root = NULL;
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 75);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 60);
    root = insert(root, 90);
    /* Example: successor of 40 is 50 */
    findSuccessor(root, 40);

    freeTree(root);
    return 0;
}
```

Question 2: Binary Tree – Count Nodes with Exactly One Child

Problem

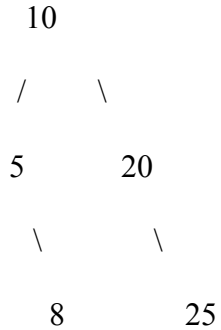
Given a binary tree, write a function to count how many nodes have **exactly one child**.

Function Prototype

```
int countSingleChildNodes(struct Node* root);
```

Example

Tree:



Nodes with exactly one child:

- 5 (only right child)
- 20 (only right child)

Output:

2

Constraints

- Must use recursion
- Do NOT use global variables

#Template

```
/* Create a new node */

struct Node* createNode(int value) { //TODO }

/* Count nodes with exactly one child */

int countSingleChildNodes(struct Node* root) {

    if (root == NULL) {

        return 0;

    }

    int count = 0;

    if ((root->left != NULL && root->right == NULL) ||

        (root->left == NULL && root->right != NULL)) {
```

```
        count = 1;
    }
    return count + countSingleChildNodes(root->left) + countSingleChildNodes(root->right);
}
/* Free tree memory */
void freeTree(struct Node* root) {
   //TODO
    }
    //TODO
}
int main() {
    struct Node* root = createNode(10);
    root->left = createNode(5);
    root->right = createNode(20);
    root->left->right = createNode(8);
    root->right->right = createNode(25);
    printf("%d\n", countSingleChildNodes(root));
    freeTree(root);
    return 0;
}
```

Question 3: BST – Delete a Node with One Child

Problem

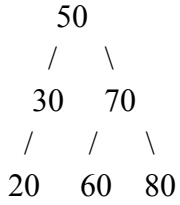
You are given a Binary Search Tree (BST).

Write a function to **delete a node that has exactly one child**.

Function Prototype

struct Node* deleteOneChild(struct Node* root, int key);

Input Example (tree already constructed)



Delete:

key = 30

Expected Output (In-order Traversal)

Before deletion:

20 30 50 60 70 80

After deletion:

20 50 60 70 80

Rules

- Use BST properties
- Do NOT use arrays
- Return updated root
- Print in-order before and after deletion

#Template

```
struct Node {
   int data;
   struct Node* left;
   struct Node* right;
};
```

```
struct Node* createNode(int value) {
  //TODO
}

struct Node* insert(struct Node* root, int value) {
 // TODO
}

struct Node* deleteOneChild(struct Node* root, int key) {
   if (root == NULL) {
      return NULL;
   }

   if (key < root->data) {
      root->left = deleteOneChild(root->left, key);
   }
   else if (key > root->data) {
      root->right = deleteOneChild(root->right, key);
   }
   else {
      /* Node found */

      /* Case: Only right child */
      if (root->left == NULL && root->right != NULL) {
         struct Node* temp = root->right;
         free(root);
         return temp;
      }

      /* Case: Only left child */
      if (root->right == NULL && root->left != NULL) {
         struct Node* temp = root->left;
         free(root);
         return temp;
      }

      /* If node has 0 or 2 children (not expected in this problem) */
      return root;
   }
```

```
    return root;
}

void inorder(struct Node* root) {
 //TODO
}

int main() {
    struct Node* root = NULL;

    /* Build the example tree */
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 20);
    root = insert(root, 60);
    root = insert(root, 80);

    printf("Before deletion: ");
    inorder(root);
    printf("\n");

    /* Delete node with one child */
    root = deleteOneChild(root, 30);

    printf("After deletion: ");
    inorder(root);
    printf("\n");

    freeTree(root);
    return 0;
}
```